

VEHICLE PARKING FINDER APP

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

Hamsini 2520030016

Hasini 2520030116

Under the Esteemed Guidance of

Dr. Ramya Krishna

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

**K L (Deemed to be) University DEPARTMENT OF COMPUTER SCIENCE
ENGINEERING**

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



The Project Report entitled “**Vehicle Parking Finder App**” is a record of Bonafide work of **Hamsini - 2520030016, Hasini – 2520030116** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

Hamsini – 2520030016

Hasini – 2520030116

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



This is certify that the mini project based report entitled “**Vehicle Parking Finder App**” is a bonafide work done and submitted by **Hamsini - 2520030016, Hasini – 2520030116** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2026-2027**.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator **Dr Yerragudipadu Subbarayudu**, and my guide **Dr Ramya Krishna**, Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Roa (CSE)**, Head of the **Department for Computer Science Engineering** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

ABSTRACT

The Vehicle Parking Finder App is a web-based parking management and booking application developed to simplify the process of finding and reserving parking for visitors of Nexus Mall. The project is intentionally focused on one parking location so that the complete workflow can be modelled in a manageable and demonstrable manner. The system provides a structured digital journey beginning with registration and login, followed by parking availability, slot selection, vehicle-detail entry, booking creation, payment-step navigation, confirmation, and booking-history access.

The frontend is developed using React.js and provides a dashboard-style user interface with separate pages and reusable components. The backend is implemented using Node.js and Express.js and exposes RESTful API routes. MySQL is used as the relational database for storing users, parking areas, parking slots, bookings, and payment-related information. When a booking is successfully created, the selected parking slot is updated from AVAILABLE to BOOKED and the available-slot count is adjusted in the associated parking area.

The current implementation represents a local full-stack prototype. The application can be extended into a production real-time parking platform through integration with physical parking sensors, a production payment gateway, notifications, administrator dashboards, authentication hardening, cloud deployment, analytics, and mobile applications. The project demonstrates the practical integration of frontend development, backend services, relational database design, API communication, and software testing in a real-world problem domain.

INDEX

S. No.	Chapter	Topics	Page No.
1	Introduction	1.1 Background; 1.2 Problem Statement; 1.3 Scope; 1.4 Objectives; 1.5 Importance; 1.6 Target Users	7–17
2	System Requirements	2.1 Hardware; 2.2 Software; 2.3 Development Tools and Frameworks	18–23
3	Technology Stack	3.1 React.js; 3.2 Node.js/Express; 3.3 MySQL; 3.4 Git/GitHub; 3.5 APIs/Services	24–34
4	System Architecture	4.1 High-level Architecture; 4.2 Layer Description; 4.3 Deployment Architecture	35–38
5	Design	5.1 UI Design; 5.2 ER Diagram/Schema; 5.3 Navigation Flow	39–40
6	Implementation	6.1 Modules; 6.2 Frontend Logic; 6.3 API Integration; 6.4 Authentication	41–50
7	Features	7.1 Main Features; 7.2 User Perspective; 7.3 Technical Description	51–58
8	Testing	8.1 API Testing; 8.2 Integration; 8.3 Tools; 8.4 Test Cases	59–67
9	Deployment	9.1 Steps; 9.2 Environment; 9.3 Hosting	68–76
10	Challenges & Limitations	10.1 Issues; 10.2 Solutions; 10.3 Limitations/Bugs	77–85

11	Future Enhancements	11.1 Planned Features; 11.2 Integrations/Optimizations	86–91
12	Conclusion	12.1 Summary; 12.2 Achievements; 12.3 Skills Learned	92–100
13	References	Books, tutorials, APIs, documentation sites	101–102
14	Appendices	Screenshots; code snippets; setup; user guide; geo-tag placeholders; review forms	103–105

CHAPTER 1 - INTRODUCTION

1.1 Background of the Project

The Vehicle Parking Finder App is developed around a practical parking problem faced by visitors of Nexus Mall. This page examines the growth of digital parking services and the practical problem of searching for parking at a busy mall. The emphasis is on translating a familiar physical activity into a structured digital workflow. Instead of requiring a visitor to depend entirely on signs, manual circulation, or uncertain information, the application provides a central interface through which the user can view parking information maintained in the project database.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

1.2 Problem Statement

The Vehicle Parking Finder App is developed around a practical parking problem faced by visitors of Nexus Mall. This page examines the fragmented manual process of entering a parking facility, checking signs, circulating between lanes, and identifying free spaces. The emphasis is on translating a familiar physical activity into a structured digital workflow. Instead of requiring a visitor to depend entirely on signs, manual circulation, or uncertain information, the application provides a central interface through which the user can view parking information maintained in the project database.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

1.3 Scope of the Project

The Vehicle Parking Finder App is developed around a practical parking problem faced by visitors of Nexus Mall. This page examines the focused scope of modelling one parking facility at Nexus Mall instead of maintaining data for every city and every parking location. The emphasis is on translating a familiar physical activity into a structured digital workflow. Instead of requiring a visitor to depend entirely on signs, manual circulation, or uncertain information, the application provides a central interface through which the user can view parking information maintained in the project database.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

1.4 Objectives of the Project

The Vehicle Parking Finder App is developed around a practical parking problem faced by visitors of Nexus Mall. This page examines the complete user journey from registration and login through availability, slot selection, vehicle details, booking, payment step, and confirmation. The emphasis is on translating a familiar physical activity into a structured digital workflow. Instead of requiring a visitor to depend entirely on signs, manual circulation, or uncertain information, the application provides a central interface through which the user can view parking information maintained in the project database.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

1.5 Importance of the Application

The Vehicle Parking Finder App is developed around a practical parking problem faced by visitors of Nexus Mall. This page examines the role of a relational database in maintaining users, parking areas, parking slots, bookings, and status information. The emphasis is on translating a familiar physical activity into a structured digital workflow. Instead of requiring a visitor to depend entirely on signs, manual circulation, or uncertain information, the application provides a central interface through which the user can view parking information maintained in the project database.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

1.5 Target Users / Audience

The Vehicle Parking Finder App is developed around a practical parking problem faced by visitors of Nexus Mall. This page examines the distinction between database-backed current availability in the prototype and future integration with physical parking sensors. The emphasis is on translating a familiar physical activity into a structured digital workflow. Instead of requiring a visitor to depend entirely on signs, manual circulation, or uncertain information, the application provides a central interface through which the user can view parking information maintained in the project database.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

1.1 Background of the Project

The Vehicle Parking Finder App is developed around a practical parking problem faced by visitors of Nexus Mall. This page examines the importance of a clear interface for visitors, parking operators, and future administrators. The emphasis is on translating a familiar physical activity into a structured digital workflow. Instead of requiring a visitor to depend entirely on signs, manual circulation, or uncertain information, the application provides a central interface through which the user can view parking information maintained in the project database.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

1.2 Problem Statement

The Vehicle Parking Finder App is developed around a practical parking problem faced by visitors of Nexus Mall. This page examines the academic value of combining frontend development, backend services, database management, API integration, and testing. The emphasis is on translating a familiar physical activity into a structured digital workflow. Instead of requiring a visitor to depend entirely on signs, manual circulation, or uncertain information, the application provides a central interface through which the user can view parking information maintained in the project database.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

1.3 Scope of the Project

The Vehicle Parking Finder App is developed around a practical parking problem faced by visitors of Nexus Mall. This page examines the target users and how the system can later be extended to administrators and parking staff. The emphasis is on translating a familiar physical activity into a structured digital workflow. Instead of requiring a visitor to depend entirely on signs, manual circulation, or uncertain information, the application provides a central interface through which the user can view parking information maintained in the project database.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

1.4 Objectives of the Project

The Vehicle Parking Finder App is developed around a practical parking problem faced by visitors of Nexus Mall. This page examines the boundaries of the current prototype, including local development and payment-step integration. The emphasis is on translating a familiar physical activity into a structured digital workflow. Instead of requiring a visitor to depend entirely on signs, manual circulation, or uncertain information, the application provides a central interface through which the user can view parking information maintained in the project database.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

1.6 Importance of the Application

The Vehicle Parking Finder App is developed around a practical parking problem faced by visitors of Nexus Mall. This page examines the overall problem-to-solution mapping for the Nexus Mall parking use case. The emphasis is on translating a familiar physical activity into a structured digital workflow. Instead of requiring a visitor to depend entirely on signs, manual circulation, or uncertain information, the application provides a central interface through which the user can view parking information maintained in the project database.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

CHAPTER 2 - SYSTEM REQUIREMENTS

2.1 Hardware Requirements

System requirements for the Vehicle Parking Finder App are defined from the perspective of the actual development environment used for the project. The application requires a machine capable of running a React development server, a Node.js/Express backend, MySQL, a browser, and development tools simultaneously. This section discusses developer hardware requirements for running VS Code, Node.js, React development servers, a browser, and MySQL Workbench simultaneously.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

Aspect	Project Treatment	Evidence / Verification
Frontend	React.js pages and reusable components	Browser testing and source code
Backend	Node.js + Express.js REST API	Server console and API responses
Database	MySQL relational schema	MySQL Workbench / SQL queries
Integration	HTTP requests from React to Express	Postman + browser network tab
Component	Minimum	Recommended
Processor	Dual-core	Intel Core i5 / Ryzen 5 or better
RAM	4 GB	8–16 GB
Storage	20 GB free	SSD with 50 GB+ free
Network	Basic broadband	Stable broadband

Browser	Modern browser	Latest Chrome / Edge / Firefox
---------	----------------	--------------------------------

2.2 Software Requirements

System requirements for the Vehicle Parking Finder App are defined from the perspective of the actual development environment used for the project. The application requires a machine capable of running a React development server, a Node.js/Express backend, MySQL, a browser, and development tools simultaneously. This section discusses recommended RAM, processor, storage, display, keyboard, mouse, and network requirements.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

Aspect	Project Treatment	Evidence / Verification
Frontend	React.js pages and reusable components	Browser testing and source code
Backend	Node.js + Express.js REST API	Server console and API responses
Database	MySQL relational schema	MySQL Workbench / SQL queries
Integration	HTTP requests from React to Express	Postman + browser network tab

2.3 Development Tools and Frameworks

System requirements for the Vehicle Parking Finder App are defined from the perspective of the actual development environment used for the project. The application requires a machine capable of running a React development server, a Node.js/Express backend, MySQL, a browser, and development tools simultaneously. This section discusses software requirements including Windows/Linux/macOS, Node.js, npm, VS Code, MySQL, MySQL Workbench, and a modern browser.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

Aspect	Project Treatment	Evidence / Verification
Frontend	React.js pages and reusable components	Browser testing and source code
Backend	Node.js + Express.js REST API	Server console and API responses
Database	MySQL relational schema	MySQL Workbench / SQL queries
Integration	HTTP requests from React to Express	Postman + browser network tab

2.1 Hardware Requirements

System requirements for the Vehicle Parking Finder App are defined from the perspective of the actual development environment used for the project. The application requires a machine capable of running a React development server, a Node.js/Express backend, MySQL, a browser, and development tools simultaneously. This section discusses the React development environment and npm package management.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

Aspect	Project Treatment	Evidence / Verification
Frontend	React.js pages and reusable components	Browser testing and source code
Backend	Node.js + Express.js REST API	Server console and API responses
Database	MySQL relational schema	MySQL Workbench / SQL queries
Integration	HTTP requests from React to Express	Postman + browser network tab

2.2 Software Requirements

System requirements for the Vehicle Parking Finder App are defined from the perspective of the actual development environment used for the project. The application requires a machine capable of running a React development server, a Node.js/Express backend, MySQL, a browser, and development tools simultaneously. This section discusses the backend runtime and required Node.js dependencies.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

Aspect	Project Treatment	Evidence / Verification
Frontend	React.js pages and reusable components	Browser testing and source code
Backend	Node.js + Express.js REST API	Server console and API responses
Database	MySQL relational schema	MySQL Workbench / SQL queries
Integration	HTTP requests from React to Express	Postman + browser network tab

2.3 Development Tools and Frameworks

System requirements for the Vehicle Parking Finder App are defined from the perspective of the actual development environment used for the project. The application requires a machine capable of running a React development server, a Node.js/Express backend, MySQL, a browser, and development tools simultaneously. This section discusses the database environment and MySQL Workbench usage for schema creation and testing.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

Aspect	Project Treatment	Evidence / Verification
Frontend	React.js pages and reusable components	Browser testing and source code
Backend	Node.js + Express.js REST API	Server console and API responses
Database	MySQL relational schema	MySQL Workbench / SQL queries
Integration	HTTP requests from React to Express	Postman + browser network tab

CHAPTER 3 - TECHNOLOGY STACK

3.1 Front-end – React.js

The technology stack of the Vehicle Parking Finder App is organised so that each technology has a clearly defined responsibility. The React frontend handles presentation and user interaction, the Node.js/Express backend exposes application services, and MySQL stores persistent relational data. This page focuses on React.js component architecture, JSX, reusable components, page routing, forms, state, and API calls.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

Aspect	Project Treatment	Evidence / Verification
Frontend	React.js pages and reusable components	Browser testing and source code
Backend	Node.js + Express.js REST API	Server console and API responses
Database	MySQL relational schema	MySQL Workbench / SQL queries
Integration	HTTP requests from React to Express	Postman + browser network tab
Layer	Technology	Responsibility
Frontend	React.js	UI rendering, routing, forms, state
Backend	Node.js / Express.js	REST APIs and server logic
Database	MySQL	Persistent relational data
Testing	Postman / Browser DevTools	API and integration verification

IDE	VS Code	Development and debugging
-----	---------	---------------------------

3.2 Back-end – Node.js and Express.js

The technology stack of the Vehicle Parking Finder App is organised so that each technology has a clearly defined responsibility. The React frontend handles presentation and user interaction, the Node.js/Express backend exposes application services, and MySQL stores persistent relational data. This page focuses on Node.js as the runtime and Express.js as the HTTP server framework.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

Aspect	Project Treatment	Evidence / Verification
Frontend	React.js pages and reusable components	Browser testing and source code
Backend	Node.js + Express.js REST API	Server console and API responses
Database	MySQL relational schema	MySQL Workbench / SQL queries
Integration	HTTP requests from React to Express	Postman + browser network tab

3.3 Database – MySQL

The technology stack of the Vehicle Parking Finder App is organised so that each technology has a clearly defined responsibility. The React frontend handles presentation and user interaction, the Node.js/Express backend exposes application services, and MySQL stores persistent relational data. This page focuses on REST API design for users, parking areas, parking slots, bookings, and payment/confirmation workflow.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

Aspect	Project Treatment	Evidence / Verification
Frontend	React.js pages and reusable components	Browser testing and source code
Backend	Node.js + Express.js REST API	Server console and API responses
Database	MySQL relational schema	MySQL Workbench / SQL queries
Integration	HTTP requests from React to Express	Postman + browser network tab

3.4 Version Control – Git and GitHub

The technology stack of the Vehicle Parking Finder App is organised so that each technology has a clearly defined responsibility. The React frontend handles presentation and user interaction, the Node.js/Express backend exposes application services, and MySQL stores persistent relational data. This page focuses on MySQL tables, primary keys, foreign keys, relationships, constraints, and SQL queries.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

Aspect	Project Treatment	Evidence / Verification
Frontend	React.js pages and reusable components	Browser testing and source code
Backend	Node.js + Express.js REST API	Server console and API responses
Database	MySQL relational schema	MySQL Workbench / SQL queries
Integration	HTTP requests from React to Express	Postman + browser network tab

3.5 APIs / External Services

The technology stack of the Vehicle Parking Finder App is organised so that each technology has a clearly defined responsibility. The React frontend handles presentation and user interaction, the Node.js/Express backend exposes application services, and MySQL stores persistent relational data. This page focuses on Git and GitHub as optional version-control practices for maintaining the project.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

Aspect	Project Treatment	Evidence / Verification
Frontend	React.js pages and reusable components	Browser testing and source code
Backend	Node.js + Express.js REST API	Server console and API responses
Database	MySQL relational schema	MySQL Workbench / SQL queries
Integration	HTTP requests from React to Express	Postman + browser network tab

3.1 Front-end – React.js

The technology stack of the Vehicle Parking Finder App is organised so that each technology has a clearly defined responsibility. The React frontend handles presentation and user interaction, the Node.js/Express backend exposes application services, and MySQL stores persistent relational data. This page focuses on environment variables, API base URLs, CORS, JSON request bodies, and local development configuration.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

Aspect	Project Treatment	Evidence / Verification
Frontend	React.js pages and reusable components	Browser testing and source code
Backend	Node.js + Express.js REST API	Server console and API responses
Database	MySQL relational schema	MySQL Workbench / SQL queries
Integration	HTTP requests from React to Express	Postman + browser network tab

3.2 Back-end – Node.js and Express.js

The technology stack of the Vehicle Parking Finder App is organised so that each technology has a clearly defined responsibility. The React frontend handles presentation and user interaction, the Node.js/Express backend exposes application services, and MySQL stores persistent relational data. This page focuses on the reason a relational database is suitable for the structured parking entities used by the project.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

Aspect	Project Treatment	Evidence / Verification
Frontend	React.js pages and reusable components	Browser testing and source code
Backend	Node.js + Express.js REST API	Server console and API responses
Database	MySQL relational schema	MySQL Workbench / SQL queries
Integration	HTTP requests from React to Express	Postman + browser network tab

3.3 Database – MySQL

The technology stack of the Vehicle Parking Finder App is organised so that each technology has a clearly defined responsibility. The React frontend handles presentation and user interaction, the Node.js/Express backend exposes application services, and MySQL stores persistent relational data. This page focuses on how the selected technologies cooperate across the application stack.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

Aspect	Project Treatment	Evidence / Verification
Frontend	React.js pages and reusable components	Browser testing and source code
Backend	Node.js + Express.js REST API	Server console and API responses
Database	MySQL relational schema	MySQL Workbench / SQL queries
Integration	HTTP requests from React to Express	Postman + browser network tab

3.4 Version Control – Git and GitHub

The technology stack of the Vehicle Parking Finder App is organised so that each technology has a clearly defined responsibility. The React frontend handles presentation and user interaction, the Node.js/Express backend exposes application services, and MySQL stores persistent relational data. This page focuses on development and debugging tools such as browser DevTools, Postman, VS Code, and MySQL Workbench.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

Aspect	Project Treatment	Evidence / Verification
Frontend	React.js pages and reusable components	Browser testing and source code
Backend	Node.js + Express.js REST API	Server console and API responses
Database	MySQL relational schema	MySQL Workbench / SQL queries
Integration	HTTP requests from React to Express	Postman + browser network tab

3.5 APIs / External Services

The technology stack of the Vehicle Parking Finder App is organised so that each technology has a clearly defined responsibility. The React frontend handles presentation and user interaction, the Node.js/Express backend exposes application services, and MySQL stores persistent relational data. This page focuses on possible external service integration for a production payment gateway and future live sensor feeds.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

Aspect	Project Treatment	Evidence / Verification
Frontend	React.js pages and reusable components	Browser testing and source code
Backend	Node.js + Express.js REST API	Server console and API responses
Database	MySQL relational schema	MySQL Workbench / SQL queries
Integration	HTTP requests from React to Express	Postman + browser network tab

3.1 Front-end – React.js

The technology stack of the Vehicle Parking Finder App is organised so that each technology has a clearly defined responsibility. The React frontend handles presentation and user interaction, the Node.js/Express backend exposes application services, and MySQL stores persistent relational data. This page focuses on security-related technology considerations for authentication, input validation, and database access.

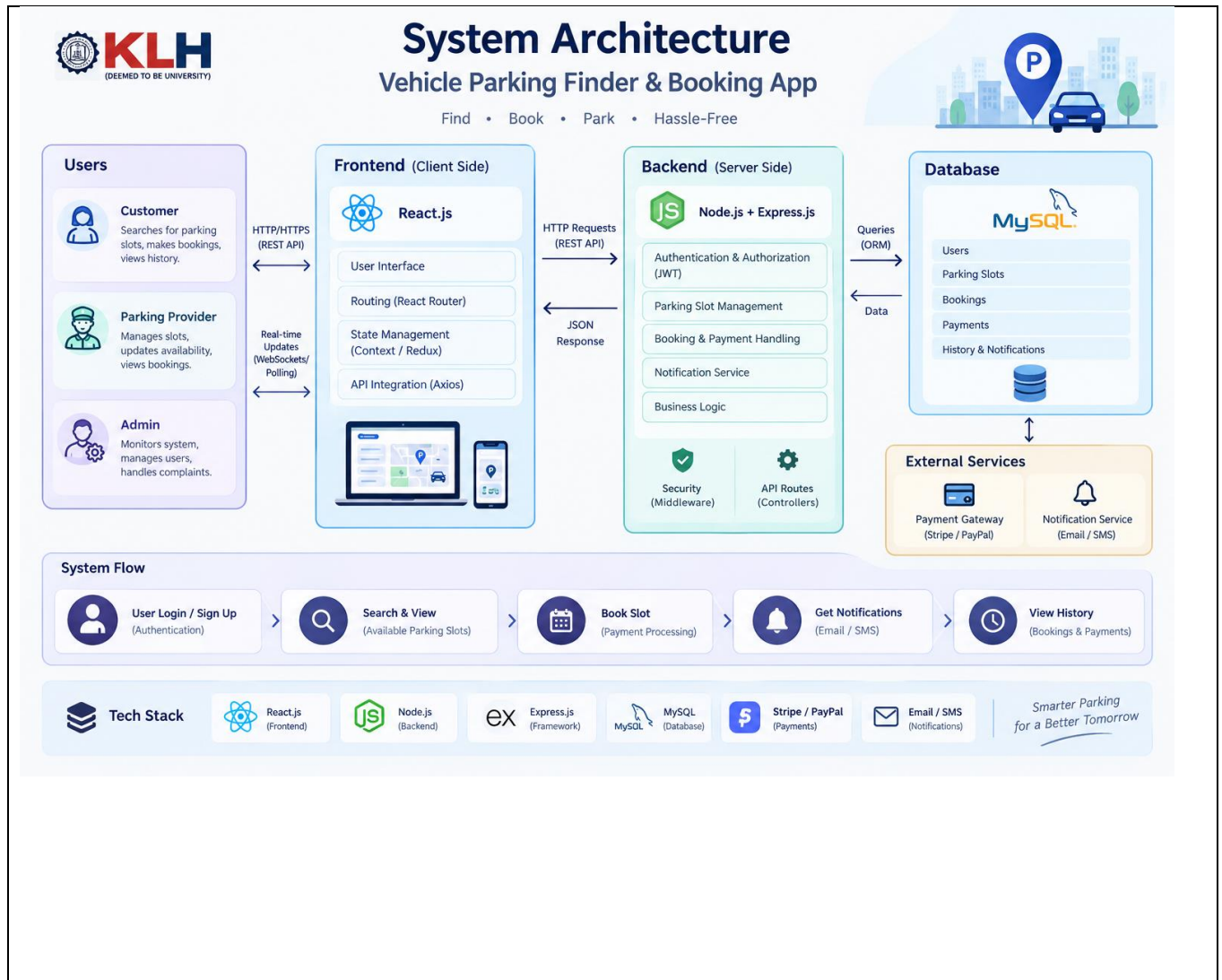
In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

Aspect	Project Treatment	Evidence / Verification
Frontend	React.js pages and reusable components	Browser testing and source code
Backend	Node.js + Express.js REST API	Server console and API responses
Database	MySQL relational schema	MySQL Workbench / SQL queries
Integration	HTTP requests from React to Express	Postman + browser network tab

CHAPTER 4 - SYSTEM ARCHITECTURE

React Frontend → REST API / Express.js → MySQL Database; external payment service and future sensor feed shown as optional future integrations.



4.1 High-level Architecture Diagram

The architecture of the Vehicle Parking Finder App follows a layered web-application model. The browser communicates with the React application, React sends HTTP requests to Express routes, Express validates and processes requests, and the database layer performs SQL operations. This page explains the high-level React-to-Express-to-MySQL request-response flow.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

The documentation is intended to be read alongside the source code, database schema, screenshots, and test evidence. Where a final project artifact is required but was not available for inclusion in this generated report, a clearly labelled placeholder has been retained. This keeps the report academically useful without presenting invented screenshots, URLs, measurements, credentials, or production integrations as completed facts.

4.2 Description of Each Layer

The architecture of the Vehicle Parking Finder App follows a layered web-application model. The browser communicates with the React application, React sends HTTP requests to Express routes, Express validates and processes requests, and the database layer performs SQL operations. This page explains the presentation layer, application/API layer, and data layer.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

The documentation is intended to be read alongside the source code, database schema, screenshots, and test evidence. Where a final project artifact is required but was not available for inclusion in this generated report, a clearly labelled placeholder has been retained. This keeps the report academically useful without presenting invented screenshots, URLs, measurements, credentials, or production integrations as completed facts.

4.3 Deployment Architecture

The architecture of the Vehicle Parking Finder App follows a layered web-application model. The browser communicates with the React application, React sends HTTP requests to Express routes, Express validates and processes requests, and the database layer performs SQL operations. This page explains how registration and login requests travel from the browser to the backend and database.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

The documentation is intended to be read alongside the source code, database schema, screenshots, and test evidence. Where a final project artifact is required but was not available for inclusion in this generated report, a clearly labelled placeholder has been retained. This keeps the report academically useful without presenting invented screenshots, URLs, measurements, credentials, or production integrations as completed facts.

4.1 High-level Architecture Diagram

The architecture of the Vehicle Parking Finder App follows a layered web-application model. The browser communicates with the React application, React sends HTTP requests to Express routes, Express validates and processes requests, and the database layer performs SQL operations. This page explains how slot availability is retrieved and how a booking changes the database state.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

The documentation is intended to be read alongside the source code, database schema, screenshots, and test evidence. Where a final project artifact is required but was not available for inclusion in this generated report, a clearly labelled placeholder has been retained. This keeps the report academically useful without presenting invented screenshots, URLs, measurements, credentials, or production integrations as completed facts.

CHAPTER 5 - DESIGN

5.1 User Interface Design

Design decisions in the Vehicle Parking Finder App are guided by the need for clarity, consistency, and a short user journey. The design covers the visual dashboard as well as the relational database structure. This page discusses the dashboard-style interface, navigation hierarchy, forms, cards, slot indicators, buttons, and responsive styling.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

The documentation is intended to be read alongside the source code, database schema, screenshots, and test evidence. Where a final project artifact is required but was not available for inclusion in this generated report, a clearly labelled placeholder has been retained. This keeps the report academically useful without presenting invented screenshots, URLs, measurements, credentials, or production integrations as completed facts.

5.2 ER Diagram / Database Schema

Design decisions in the Vehicle Parking Finder App are guided by the need for clarity, consistency, and a short user journey. The design covers the visual dashboard as well as the relational database structure. This page discusses the relational database design and relationships between users, parking areas, parking slots, bookings, and payment information.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

The documentation is intended to be read alongside the source code, database schema, screenshots, and test evidence. Where a final project artifact is required but was not available for inclusion in this generated report, a clearly labelled placeholder has been retained. This keeps the report academically useful without presenting invented screenshots, URLs, measurements, credentials, or production integrations as completed facts.

CHAPTER 6 - IMPLEMENTATION

6.1 Module-wise Implementation

Implementation converts the requirements and design into executable modules. In the Vehicle Parking Finder App, the frontend is organised into pages and reusable components while the backend is organised into server configuration, database connectivity, and route modules. This page focuses on project folder structure and separation of components, pages, services, assets, routes, and backend modules.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

1. The user registers or logs in before accessing the protected booking journey.
2. The frontend requests parking information from the backend and renders the available slots.
3. The user selects a slot and submits vehicle and booking information.
4. The backend validates the slot, creates the booking, and updates slot status.
5. The frontend redirects the user to the payment step and then to confirmation.

6.2 Front-end Logic

Implementation converts the requirements and design into executable modules. In the Vehicle Parking Finder App, the frontend is organised into pages and reusable components while the backend is organised into server configuration, database connectivity, and route modules. This page focuses on registration and login implementation.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

6. The user registers or logs in before accessing the protected booking journey.
7. The frontend requests parking information from the backend and renders the available slots.
8. The user selects a slot and submits vehicle and booking information.
9. The backend validates the slot, creates the booking, and updates slot status.
10. The frontend redirects the user to the payment step and then to confirmation.

6.3 API Integration

Implementation converts the requirements and design into executable modules. In the Vehicle Parking Finder App, the frontend is organised into pages and reusable components while the backend is organised into server configuration, database connectivity, and route modules. This page focuses on parking-area and slot display implementation.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

11. The user registers or logs in before accessing the protected booking journey.
12. The frontend requests parking information from the backend and renders the available slots.
13. The user selects a slot and submits vehicle and booking information.
14. The backend validates the slot, creates the booking, and updates slot status.
15. The frontend redirects the user to the payment step and then to confirmation.

6.4 Authentication and Authorization

Implementation converts the requirements and design into executable modules. In the Vehicle Parking Finder App, the frontend is organised into pages and reusable components while the backend is organised into server configuration, database connectivity, and route modules. This page focuses on slot selection and vehicle-detail collection.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

16. The user registers or logs in before accessing the protected booking journey.
17. The frontend requests parking information from the backend and renders the available slots.
18. The user selects a slot and submits vehicle and booking information.
19. The backend validates the slot, creates the booking, and updates slot status.
20. The frontend redirects the user to the payment step and then to confirmation.

6.1 Module-wise Implementation

Implementation converts the requirements and design into executable modules. In the Vehicle Parking Finder App, the frontend is organised into pages and reusable components while the backend is organised into server configuration, database connectivity, and route modules. This page focuses on booking API implementation and validation.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

21. The user registers or logs in before accessing the protected booking journey.
22. The frontend requests parking information from the backend and renders the available slots.
23. The user selects a slot and submits vehicle and booking information.
24. The backend validates the slot, creates the booking, and updates slot status.
25. The frontend redirects the user to the payment step and then to confirmation.

[INSERT MODULE / FLOW DIAGRAM HERE]

Registration → Login → Parking Dashboard → Slot Selection → Vehicle Details → Booking → Payment → Confirmation

6.2 Front-end Logic

Implementation converts the requirements and design into executable modules. In the Vehicle Parking Finder App, the frontend is organised into pages and reusable components while the backend is organised into server configuration, database connectivity, and route modules. This page focuses on payment-step and confirmation workflow.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

26. The user registers or logs in before accessing the protected booking journey.
27. The frontend requests parking information from the backend and renders the available slots.
28. The user selects a slot and submits vehicle and booking information.
29. The backend validates the slot, creates the booking, and updates slot status.
30. The frontend redirects the user to the payment step and then to confirmation.

6.3 API Integration

Implementation converts the requirements and design into executable modules. In the Vehicle Parking Finder App, the frontend is organised into pages and reusable components while the backend is organised into server configuration, database connectivity, and route modules. This page focuses on booking-history implementation.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

31. The user registers or logs in before accessing the protected booking journey.
32. The frontend requests parking information from the backend and renders the available slots.
33. The user selects a slot and submits vehicle and booking information.
34. The backend validates the slot, creates the booking, and updates slot status.
35. The frontend redirects the user to the payment step and then to confirmation.

6.4 Authentication and Authorization

Implementation converts the requirements and design into executable modules. In the Vehicle Parking Finder App, the frontend is organised into pages and reusable components while the backend is organised into server configuration, database connectivity, and route modules. This page focuses on frontend state management and local data transfer between pages.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

36. The user registers or logs in before accessing the protected booking journey.
37. The frontend requests parking information from the backend and renders the available slots.
38. The user selects a slot and submits vehicle and booking information.
39. The backend validates the slot, creates the booking, and updates slot status.
40. The frontend redirects the user to the payment step and then to confirmation.

6.1 Module-wise Implementation

Implementation converts the requirements and design into executable modules. In the Vehicle Parking Finder App, the frontend is organised into pages and reusable components while the backend is organised into server configuration, database connectivity, and route modules. This page focuses on REST API integration and error handling.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

41. The user registers or logs in before accessing the protected booking journey.
42. The frontend requests parking information from the backend and renders the available slots.
43. The user selects a slot and submits vehicle and booking information.
44. The backend validates the slot, creates the booking, and updates slot status.
45. The frontend redirects the user to the payment step and then to confirmation.

6.2 Front-end Logic

Implementation converts the requirements and design into executable modules. In the Vehicle Parking Finder App, the frontend is organised into pages and reusable components while the backend is organised into server configuration, database connectivity, and route modules. This page focuses on authentication and authorization considerations.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

46. The user registers or logs in before accessing the protected booking journey.
47. The frontend requests parking information from the backend and renders the available slots.
48. The user selects a slot and submits vehicle and booking information.
49. The backend validates the slot, creates the booking, and updates slot status.
50. The frontend redirects the user to the payment step and then to confirmation.

CHAPTER 7 - FEATURES

7.1 List of Main Features

The Vehicle Parking Finder App provides a sequence of user-facing features rather than isolated screens. Each feature contributes to the parking journey, from account creation through slot booking and confirmation. This page explains registration and login from both the visitor and technical perspectives.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

7.2 How Each Feature Works – User Perspective

The Vehicle Parking Finder App provides a sequence of user-facing features rather than isolated screens. Each feature contributes to the parking journey, from account creation through slot booking and confirmation. This page explains Nexus Mall parking dashboard from both the visitor and technical perspectives.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

7.3 Technical Description of Features

The Vehicle Parking Finder App provides a sequence of user-facing features rather than isolated screens. Each feature contributes to the parking journey, from account creation through slot booking and confirmation. This page explains parking availability and slot status from both the visitor and technical perspectives.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

7.1 List of Main Features

The Vehicle Parking Finder App provides a sequence of user-facing features rather than isolated screens. Each feature contributes to the parking journey, from account creation through slot booking and confirmation. This page explains slot selection from both the visitor and technical perspectives.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

7.2 How Each Feature Works – User Perspective

The Vehicle Parking Finder App provides a sequence of user-facing features rather than isolated screens. Each feature contributes to the parking journey, from account creation through slot booking and confirmation. This page explains vehicle details from both the visitor and technical perspectives.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

7.3 Technical Description of Features

The Vehicle Parking Finder App provides a sequence of user-facing features rather than isolated screens. Each feature contributes to the parking journey, from account creation through slot booking and confirmation. This page explains booking creation and slot status update from both the visitor and technical perspectives.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

7.1 List of Main Features

The Vehicle Parking Finder App provides a sequence of user-facing features rather than isolated screens. Each feature contributes to the parking journey, from account creation through slot booking and confirmation. This page explains payment-step navigation from both the visitor and technical perspectives.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

7.2 How Each Feature Works – User Perspective

The Vehicle Parking Finder App provides a sequence of user-facing features rather than isolated screens. Each feature contributes to the parking journey, from account creation through slot booking and confirmation. This page explains confirmation and booking reference from both the visitor and technical perspectives.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

CHAPTER 8 - TESTING

8.1 Postman / API Testing

Testing is used to verify that the Vehicle Parking Finder App behaves as expected at the API, database, and integrated application levels. This page focuses on API endpoint testing using Postman. Test evidence should be supplemented with the actual screenshots and measured results produced during the student's final testing cycle.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

Aspect	Project Treatment	Evidence / Verification
Frontend	React.js pages and reusable components	Browser testing and source code
Backend	Node.js + Express.js REST API	Server console and API responses
Database	MySQL relational schema	MySQL Workbench / SQL queries
Integration	HTTP requests from React to Express	Postman + browser network tab

8.2 Integration Testing

Testing is used to verify that the Vehicle Parking Finder App behaves as expected at the API, database, and integrated application levels. This page focuses on database connectivity testing through the `/api/test-db` endpoint. Test evidence should be supplemented with the actual screenshots and measured results produced during the student's final testing cycle.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

Aspect	Project Treatment	Evidence / Verification
Frontend	React.js pages and reusable components	Browser testing and source code
Backend	Node.js + Express.js REST API	Server console and API responses
Database	MySQL relational schema	MySQL Workbench / SQL queries
Integration	HTTP requests from React to Express	Postman + browser network tab

8.3 Tools Used for Testing

Testing is used to verify that the Vehicle Parking Finder App behaves as expected at the API, database, and integrated application levels. This page focuses on registration and login test cases. Test evidence should be supplemented with the actual screenshots and measured results produced during the student's final testing cycle.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

Aspect		Project Treatment	Evidence / Verification
Frontend		React.js pages and reusable components	Browser testing and source code
Backend		Node.js + Express.js REST API	Server console and API responses
Database		MySQL relational schema	MySQL Workbench / SQL queries
Integration		HTTP requests from React to Express	Postman + browser network tab
Test ID	Scenario	Expected Result	Status
TC-01	Database connectivity	API returns success	To be filled
TC-02	Valid registration	User record created	To be filled
TC-03	Valid login	User authenticated	To be filled
TC-04	Available slot booking	Booking created and slot becomes BOOKED	To be filled
TC-05	Unavailable slot booking	Booking rejected	To be filled

TC-06	Booking history	User bookings returned	To be filled
-------	-----------------	---------------------------	--------------

8.4 Test Cases and Results

Testing is used to verify that the Vehicle Parking Finder App behaves as expected at the API, database, and integrated application levels. This page focuses on parking-area and parking-slot retrieval tests. Test evidence should be supplemented with the actual screenshots and measured results produced during the student's final testing cycle.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

Aspect	Project Treatment	Evidence / Verification
Frontend	React.js pages and reusable components	Browser testing and source code
Backend	Node.js + Express.js REST API	Server console and API responses
Database	MySQL relational schema	MySQL Workbench / SQL queries
Integration	HTTP requests from React to Express	Postman + browser network tab

8.1 Postman / API Testing

Testing is used to verify that the Vehicle Parking Finder App behaves as expected at the API, database, and integrated application levels. This page focuses on booking success and failure cases. Test evidence should be supplemented with the actual screenshots and measured results produced during the student's final testing cycle.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

Aspect	Project Treatment	Evidence / Verification
Frontend	React.js pages and reusable components	Browser testing and source code
Backend	Node.js + Express.js REST API	Server console and API responses
Database	MySQL relational schema	MySQL Workbench / SQL queries
Integration	HTTP requests from React to Express	Postman + browser network tab

8.2 Integration Testing

Testing is used to verify that the Vehicle Parking Finder App behaves as expected at the API, database, and integrated application levels. This page focuses on unavailable-slot validation. Test evidence should be supplemented with the actual screenshots and measured results produced during the student's final testing cycle.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

Aspect		Project Treatment	Evidence / Verification
Frontend		React.js pages and reusable components	Browser testing and source code
Backend		Node.js + Express.js REST API	Server console and API responses
Database		MySQL relational schema	MySQL Workbench / SQL queries
Integration		HTTP requests from React to Express	Postman + browser network tab
Test ID	Scenario	Expected Result	Status
TC-01	Database connectivity	API returns success	To be filled
TC-02	Valid registration	User record created	To be filled
TC-03	Valid login	User authenticated	To be filled
TC-04	Available slot booking	Booking created and slot becomes BOOKED	To be filled
TC-05	Unavailable slot booking	Booking rejected	To be filled

TC-06	Booking history	User bookings returned	To be filled
-------	-----------------	---------------------------	--------------

8.3 Tools Used for Testing

Testing is used to verify that the Vehicle Parking Finder App behaves as expected at the API, database, and integrated application levels. This page focuses on booking-history retrieval. Test evidence should be supplemented with the actual screenshots and measured results produced during the student's final testing cycle.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

Aspect	Project Treatment	Evidence / Verification
Frontend	React.js pages and reusable components	Browser testing and source code
Backend	Node.js + Express.js REST API	Server console and API responses
Database	MySQL relational schema	MySQL Workbench / SQL queries
Integration	HTTP requests from React to Express	Postman + browser network tab

8.4 Test Cases and Results

Testing is used to verify that the Vehicle Parking Finder App behaves as expected at the API, database, and integrated application levels. This page focuses on confirmation retrieval. Test evidence should be supplemented with the actual screenshots and measured results produced during the student's final testing cycle.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

Aspect	Project Treatment	Evidence / Verification
Frontend	React.js pages and reusable components	Browser testing and source code
Backend	Node.js + Express.js REST API	Server console and API responses
Database	MySQL relational schema	MySQL Workbench / SQL queries
Integration	HTTP requests from React to Express	Postman + browser network tab

8.1 Postman / API Testing

Testing is used to verify that the Vehicle Parking Finder App behaves as expected at the API, database, and integrated application levels. This page focuses on frontend-backend integration testing. Test evidence should be supplemented with the actual screenshots and measured results produced during the student's final testing cycle.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

Aspect	Project Treatment	Evidence / Verification
Frontend	React.js pages and reusable components	Browser testing and source code
Backend	Node.js + Express.js REST API	Server console and API responses
Database	MySQL relational schema	MySQL Workbench / SQL queries
Integration	HTTP requests from React to Express	Postman + browser network tab

CHAPTER 9 - DEPLOYMENT

9.1 Steps to Deploy

Deployment for the current Vehicle Parking Finder App is primarily a local development workflow, while the report also describes how the same architecture can be prepared for hosted deployment. This page focuses on local frontend and backend setup.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

Aspect	Project Treatment	Evidence / Verification
Frontend	React.js pages and reusable components	Browser testing and source code
Backend	Node.js + Express.js REST API	Server console and API responses
Database	MySQL relational schema	MySQL Workbench / SQL queries
Integration	HTTP requests from React to Express	Postman + browser network tab

51. Start MySQL and confirm the project database exists in MySQL Workbench.
52. Open the backend folder in VS Code and install Node.js dependencies.
53. Configure the database connection and environment variables.
54. Run the Express server and verify the root route and database-test route.
55. Open the frontend folder and start the React development server.
56. Verify that the frontend API base URL points to the running backend.

9.2 Environment Configuration

Deployment for the current Vehicle Parking Finder App is primarily a local development workflow, while the report also describes how the same architecture can be prepared for hosted deployment. This page focuses on environment variables and API configuration.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

Aspect	Project Treatment	Evidence / Verification
Frontend	React.js pages and reusable components	Browser testing and source code
Backend	Node.js + Express.js REST API	Server console and API responses
Database	MySQL relational schema	MySQL Workbench / SQL queries
Integration	HTTP requests from React to Express	Postman + browser network tab

9.3 Hosting of Frontend and Backend

Deployment for the current Vehicle Parking Finder App is primarily a local development workflow, while the report also describes how the same architecture can be prepared for hosted deployment. This page focuses on starting MySQL, backend server, and React development server.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

Aspect	Project Treatment	Evidence / Verification
Frontend	React.js pages and reusable components	Browser testing and source code
Backend	Node.js + Express.js REST API	Server console and API responses
Database	MySQL relational schema	MySQL Workbench / SQL queries
Integration	HTTP requests from React to Express	Postman + browser network tab

9.1 Steps to Deploy

Deployment for the current Vehicle Parking Finder App is primarily a local development workflow, while the report also describes how the same architecture can be prepared for hosted deployment. This page focuses on frontend-backend CORS configuration.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

Aspect	Project Treatment	Evidence / Verification
Frontend	React.js pages and reusable components	Browser testing and source code
Backend	Node.js + Express.js REST API	Server console and API responses
Database	MySQL relational schema	MySQL Workbench / SQL queries
Integration	HTTP requests from React to Express	Postman + browser network tab

9.2 Environment Configuration

Deployment for the current Vehicle Parking Finder App is primarily a local development workflow, while the report also describes how the same architecture can be prepared for hosted deployment. This page focuses on production deployment considerations.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

Aspect	Project Treatment	Evidence / Verification
Frontend	React.js pages and reusable components	Browser testing and source code
Backend	Node.js + Express.js REST API	Server console and API responses
Database	MySQL relational schema	MySQL Workbench / SQL queries
Integration	HTTP requests from React to Express	Postman + browser network tab

9.3 Hosting of Frontend and Backend

Deployment for the current Vehicle Parking Finder App is primarily a local development workflow, while the report also describes how the same architecture can be prepared for hosted deployment. This page focuses on database hosting considerations.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

Aspect	Project Treatment	Evidence / Verification
Frontend	React.js pages and reusable components	Browser testing and source code
Backend	Node.js + Express.js REST API	Server console and API responses
Database	MySQL relational schema	MySQL Workbench / SQL queries
Integration	HTTP requests from React to Express	Postman + browser network tab

9.1 Steps to Deploy

Deployment for the current Vehicle Parking Finder App is primarily a local development workflow, while the report also describes how the same architecture can be prepared for hosted deployment. This page focuses on frontend hosting considerations.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

Aspect	Project Treatment	Evidence / Verification
Frontend	React.js pages and reusable components	Browser testing and source code
Backend	Node.js + Express.js REST API	Server console and API responses
Database	MySQL relational schema	MySQL Workbench / SQL queries
Integration	HTTP requests from React to Express	Postman + browser network tab

9.2 Environment Configuration

Deployment for the current Vehicle Parking Finder App is primarily a local development workflow, while the report also describes how the same architecture can be prepared for hosted deployment. This page focuses on backend hosting considerations.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

Aspect	Project Treatment	Evidence / Verification
Frontend	React.js pages and reusable components	Browser testing and source code
Backend	Node.js + Express.js REST API	Server console and API responses
Database	MySQL relational schema	MySQL Workbench / SQL queries
Integration	HTTP requests from React to Express	Postman + browser network tab

9.3 Hosting of Frontend and Backend

Deployment for the current Vehicle Parking Finder App is primarily a local development workflow, while the report also describes how the same architecture can be prepared for hosted deployment. This page focuses on security and environment separation.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

Aspect	Project Treatment	Evidence / Verification
Frontend	React.js pages and reusable components	Browser testing and source code
Backend	Node.js + Express.js REST API	Server console and API responses
Database	MySQL relational schema	MySQL Workbench / SQL queries
Integration	HTTP requests from React to Express	Postman + browser network tab

CHAPTER 10 - CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

During development of the Vehicle Parking Finder App, issues can arise at the boundaries between React, Express, HTTP routes, SQL queries, and browser navigation. This page documents blank React pages and incorrect component imports and describes the corresponding debugging approach.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

10.2 Solutions Applied

During development of the Vehicle Parking Finder App, issues can arise at the boundaries between React, Express, HTTP routes, SQL queries, and browser navigation. This page documents browser console errors and route mismatches and describes the corresponding debugging approach.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

10.3 Current Limitations or Known Bugs

During development of the Vehicle Parking Finder App, issues can arise at the boundaries between React, Express, HTTP routes, SQL queries, and browser navigation. This page documents backend /api path and localhost port confusion and describes the corresponding debugging approach.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

A recurring integration issue in this type of application is confusing the backend root URL with an API route. A server may correctly respond at `http://localhost:5000/` while a route such as `/api/test-db` is available only at the complete path `http://localhost:5000/api/test-db`. Debugging therefore begins by checking the Express route definitions, server port, browser URL, and frontend request URL separately.

10.1 Issues Faced During Development

During development of the Vehicle Parking Finder App, issues can arise at the boundaries between React, Express, HTTP routes, SQL queries, and browser navigation. This page documents database connection failures and old table/schema confusion and describes the corresponding debugging approach.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

10.2 Solutions Applied

During development of the Vehicle Parking Finder App, issues can arise at the boundaries between React, Express, HTTP routes, SQL queries, and browser navigation. This page documents booking failures caused by request-body or database-schema mismatches and describes the corresponding debugging approach.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

Booking failures are commonly caused by mismatched field names between the React request body, Express destructuring, and MySQL column names. The project booking route expects values such as `user_id`, `slot_id`, `vehicle_number`, `booking_date`, `start_time`, `end_time`, and `total_amount`. A systematic check of the browser request payload, Express console output, SQL schema, and foreign-key relationships is therefore required before changing the code.

10.3 Current Limitations or Known Bugs

During development of the Vehicle Parking Finder App, issues can arise at the boundaries between React, Express, HTTP routes, SQL queries, and browser navigation. This page documents slot availability and status-update problems and describes the corresponding debugging approach.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

10.1 Issues Faced During Development

During development of the Vehicle Parking Finder App, issues can arise at the boundaries between React, Express, HTTP routes, SQL queries, and browser navigation. This page documents confirmation route errors and describes the corresponding debugging approach.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

10.2 Solutions Applied

During development of the Vehicle Parking Finder App, issues can arise at the boundaries between React, Express, HTTP routes, SQL queries, and browser navigation. This page documents frontend-backend integration debugging and describes the corresponding debugging approach.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

10.3 Current Limitations or Known Bugs

During development of the Vehicle Parking Finder App, issues can arise at the boundaries between React, Express, HTTP routes, SQL queries, and browser navigation. This page documents limitations of a single-location prototype and describes the corresponding debugging approach.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

CHAPTER 11 - FUTURE ENHANCEMENTS

11.1 Planned Features

Future development of the Vehicle Parking Finder App can extend the current prototype into a more automated parking-management platform. This page focuses on live parking sensors and IoT integration and distinguishes planned enhancements from functionality already implemented.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

Enhancement	Purpose	Current Status
IoT slot sensors	Automatically reflect physical occupancy	Future
WebSockets	Push availability changes instantly	Future
Production payment gateway	Secure real transactions	Future
Admin dashboard	Manage slots and bookings	Future
Mobile app	Native visitor experience	Future
Analytics	Occupancy and demand reporting	Future

11.2 Possible Integrations or Optimizations

Future development of the Vehicle Parking Finder App can extend the current prototype into a more automated parking-management platform. This page focuses on real-time WebSocket or Server-Sent Events updates and distinguishes planned enhancements from functionality already implemented.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

11.1 Planned Features

Future development of the Vehicle Parking Finder App can extend the current prototype into a more automated parking-management platform. This page focuses on production payment gateway integration and distinguishes planned enhancements from functionality already implemented.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

11.2 Possible Integrations or Optimizations

Future development of the Vehicle Parking Finder App can extend the current prototype into a more automated parking-management platform. This page focuses on QR-based entry and exit and distinguishes planned enhancements from functionality already implemented.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

11.1 Planned Features

Future development of the Vehicle Parking Finder App can extend the current prototype into a more automated parking-management platform. This page focuses on vehicle number recognition and distinguishes planned enhancements from functionality already implemented.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

11.2 Possible Integrations or Optimizations

Future development of the Vehicle Parking Finder App can extend the current prototype into a more automated parking-management platform. This page focuses on admin dashboard and distinguishes planned enhancements from functionality already implemented.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

CHAPTER 12 - CONCLUSION

12.1 Summary of the Project

The project outcome is best understood by comparing the original parking problem with the implemented digital workflow. This page focuses on summary of the problem and the implemented solution and records the practical learning achieved during development.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

Against the stated project objectives, the implemented prototype demonstrates the complete application flow within a single-location scope. The main achievement is not merely a collection of screens, but the integration of a React interface, Express APIs, and MySQL data so that a booking can alter the recorded parking-slot state and subsequently appear in the user's booking history.

12.2 What Was Achieved

The project outcome is best understood by comparing the original parking problem with the implemented digital workflow. This page focuses on how the frontend, backend, and database were integrated and records the practical learning achieved during development.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

12.3 Skills Learned During Development

The project outcome is best understood by comparing the original parking problem with the implemented digital workflow. This page focuses on the achieved booking workflow and records the practical learning achieved during development.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

12.1 Summary of the Project

The project outcome is best understood by comparing the original parking problem with the implemented digital workflow. This page focuses on database-driven slot status management and records the practical learning achieved during development.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

12.2 What Was Achieved

The project outcome is best understood by comparing the original parking problem with the implemented digital workflow. This page focuses on academic and practical outcomes and records the practical learning achieved during development.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

12.3 Skills Learned During Development

The project outcome is best understood by comparing the original parking problem with the implemented digital workflow. This page focuses on skills gained in React, Express, SQL, debugging, API testing, and documentation and records the practical learning achieved during development.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

12.1 Summary of the Project

The project outcome is best understood by comparing the original parking problem with the implemented digital workflow. This page focuses on known boundaries and future opportunities and records the practical learning achieved during development.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

12.2 What Was Achieved

The project outcome is best understood by comparing the original parking problem with the implemented digital workflow. This page focuses on overall project learning and conclusion and records the practical learning achieved during development.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

12.3 Skills Learned During Development

The project outcome is best understood by comparing the original parking problem with the implemented digital workflow. This page focuses on final assessment of the prototype against its stated objectives and records the practical learning achieved during development.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

- Primary context: Nexus Mall parking facility.
- Primary user: a visitor who wants to identify and reserve a parking slot.
- Core data entities: users, parking areas, parking slots, bookings, and payment-related records.
- Core state transition: AVAILABLE → BOOKED after successful booking.
- Current prototype boundary: local full-stack application with database-backed availability.

CHAPTER 13 - REFERENCES

Books, Tutorials, APIs, Documentation Sites Used

Source	Use in Project	Reference
React.js	Frontend library and component model	https://react.dev/
Node.js	JavaScript runtime for backend services	https://nodejs.org/
Express.js	Web framework for REST APIs	https://expressjs.com/
MySQL	Relational database management system	https://dev.mysql.com/doc/

The final report should also include any classroom material, database design references, UI/UX references, tutorials, API documentation, or library documentation actually consulted by the project team. The list above is a technical starting point and should not be represented as an exhaustive record of every resource used unless verified by the team.

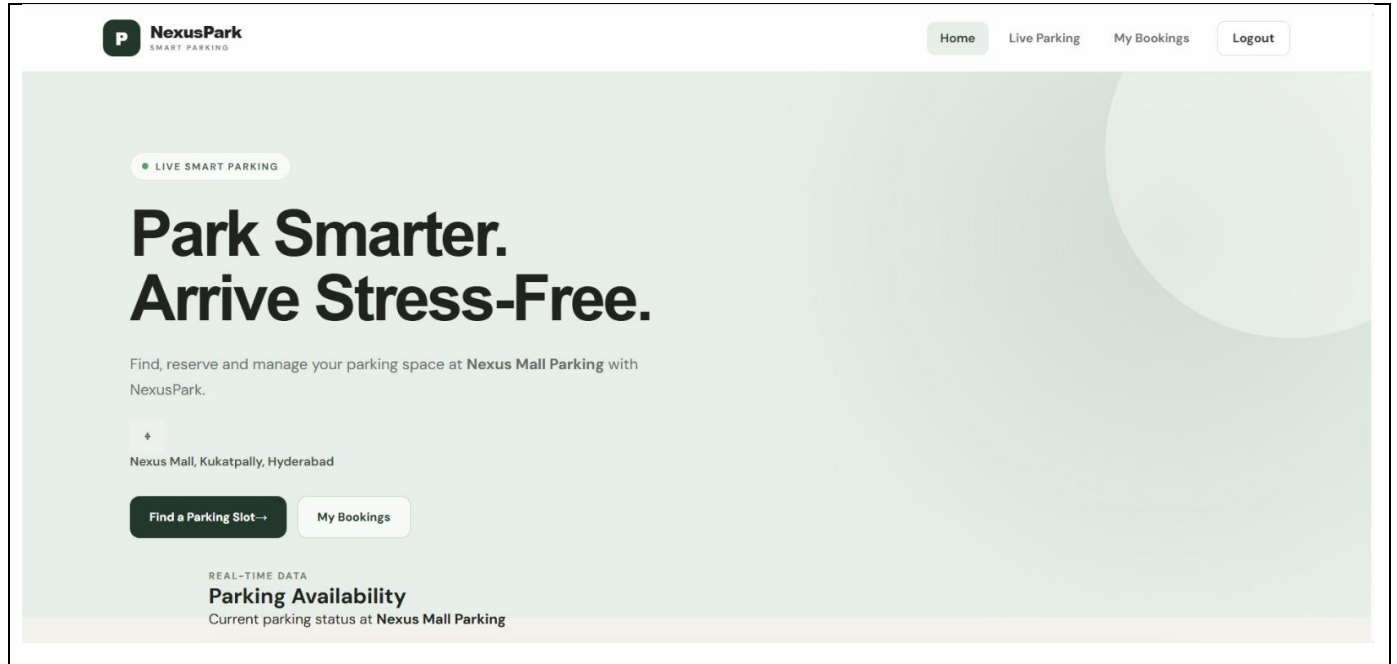
Books, Tutorials, APIs, Documentation Sites Used

Source	Use in Project	Reference
MDN Web Docs	Browser, JavaScript, HTTP, HTML and CSS references	https://developer.mozilla.org/
Git	Version control documentation	https://git-scm.com/doc
Postman	API testing and documentation	https://learning.postman.com/
VS Code	Development environment documentation	https://code.visualstudio.com/docs

The final report should also include any classroom material, database design references, UI/UX references, tutorials, API documentation, or library documentation actually consulted by the project team. The list above is a technical starting point and should not be represented as an exhaustive record of every resource used unless verified by the team.

CHAPTER 14 - APPENDICES

T



Screenshots of the App

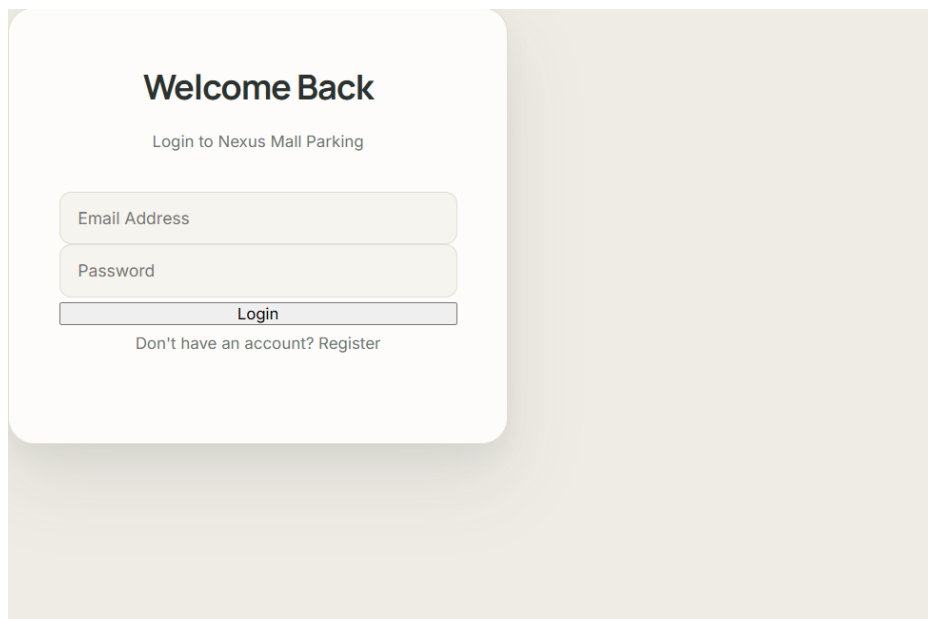
The appendices preserve implementation evidence for the Vehicle Parking Finder App. This page focuses on frontend screenshots that should be inserted from the completed application. Screenshots, code excerpts, database evidence, and review documents should be replaced or supplemented with the team's final project evidence.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

The documentation is intended to be read alongside the source code, database schema, screenshots, and test evidence. Where a final project artifact is required but was not available for inclusion in this generated report, a clearly labelled placeholder has been retained. This keeps the report academically useful without presenting invented screenshots, URLs, measurements, credentials, or production integrations as completed facts.

Registration / Login page



Sample Code Snippets

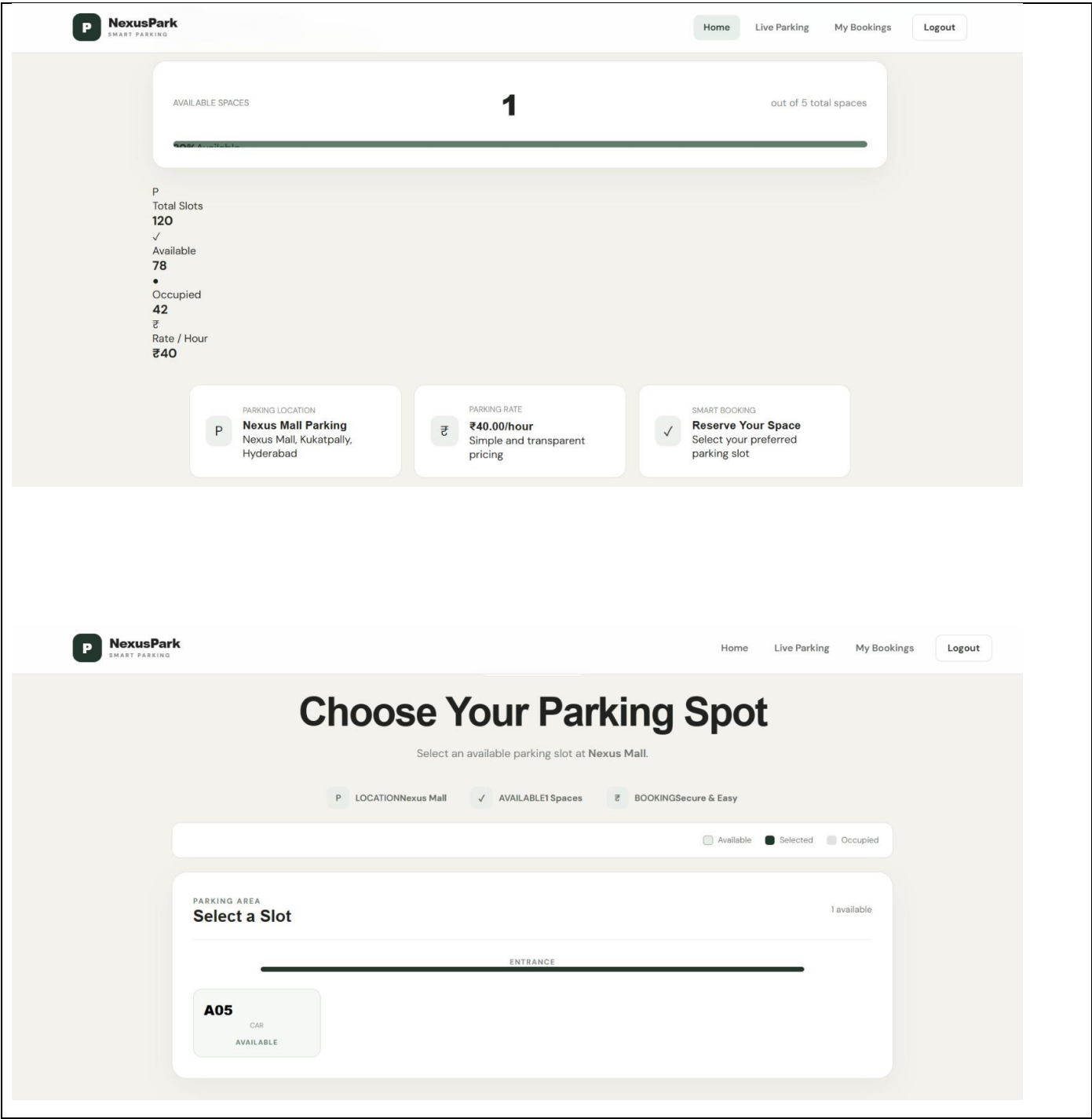
The appendices preserve implementation evidence for the Vehicle Parking Finder App. This page focuses on backend API screenshots and Postman responses. Screenshots, code excerpts, database evidence, and review documents should be replaced or supplemented with the team's final project evidence.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

The documentation is intended to be read alongside the source code, database schema, screenshots, and test evidence. Where a final project artifact is required but was not available for inclusion in this generated report, a clearly labelled placeholder has been retained. This keeps the report academically useful without presenting invented screenshots, URLs, measurements, credentials, or production integrations as completed facts.

Parking dashboard and slot availability



Installation / Setup Instructions

The appendices preserve implementation evidence for the Vehicle Parking Finder App. This page focuses on MySQL Workbench screenshots and DESCRIBE/table evidence. Screenshots, code excerpts, database evidence, and review documents should be replaced or supplemented with the team's final project evidence.

In the implemented prototype, the selected parking location is Nexus Mall. The application is not presented as a universal parking directory; its purpose is to demonstrate a complete and coherent workflow for one manageable real-world facility. The database can represent parking areas and individual slots, while the frontend translates those records into understandable cards, status indicators, forms, and navigation steps. This focused scope reduces unnecessary complexity and makes it possible to test the core flow end to end.

A key design principle is that the system should not claim information that the underlying data source does not provide. At present, the prototype uses the MySQL database as the source of parking status. A future production version can connect this model to sensors or an operator dashboard so that physical occupancy is reflected automatically. Similarly, the payment page represents the application's payment stage; a production deployment would require a verified external payment provider and secure server-side payment confirmation.

The documentation is intended to be read alongside the source code, database schema, screenshots, and test evidence. Where a final project artifact is required but was not available for inclusion in this generated report, a clearly labelled placeholder has been retained. This keeps the report academically useful without presenting invented screenshots, URLs, measurements, credentials, or production integrations as completed facts.

Booking form

